



Programa de Residência em Microeletrônica
MCTI / Softex / IRede

CONTROLADOR DE VENDING MACHINE EM SYSTEMVERILOG

Relatório técnico de projeto, simulação e síntese

Aluno	Ubajara Azevedo de Assis Filho
Trilha	RTL Design
Módulo	Projeto de Controlador Digital
Repositório	https://github.com/ubajara/Machine.git

2026

1. Introdução e objetivos

O projeto implementa o controlador digital de uma máquina de vendas com quatro produtos: café, água, suco e snack. O circuito reúne uma máquina de estados finitos de Moore, uma memória síncrona para preços e estoques, um acumulador de crédito e blocos combinacionais de comparação e subtração. O desenvolvimento foi realizado em SystemVerilog, com simulação no Synopsys VCS/Verdi e síntese no Design Compiler.

O objetivo principal foi obter um design síncrono, modular e verificável, capaz de acumular moedas, validar crédito e estoque, liberar o produto, calcular o troco, tratar cancelamentos e sinalizar erros. O testbench foi construído de forma self-checking e a síntese foi explorada para diferentes períodos de clock.

2. Arquitetura do sistema

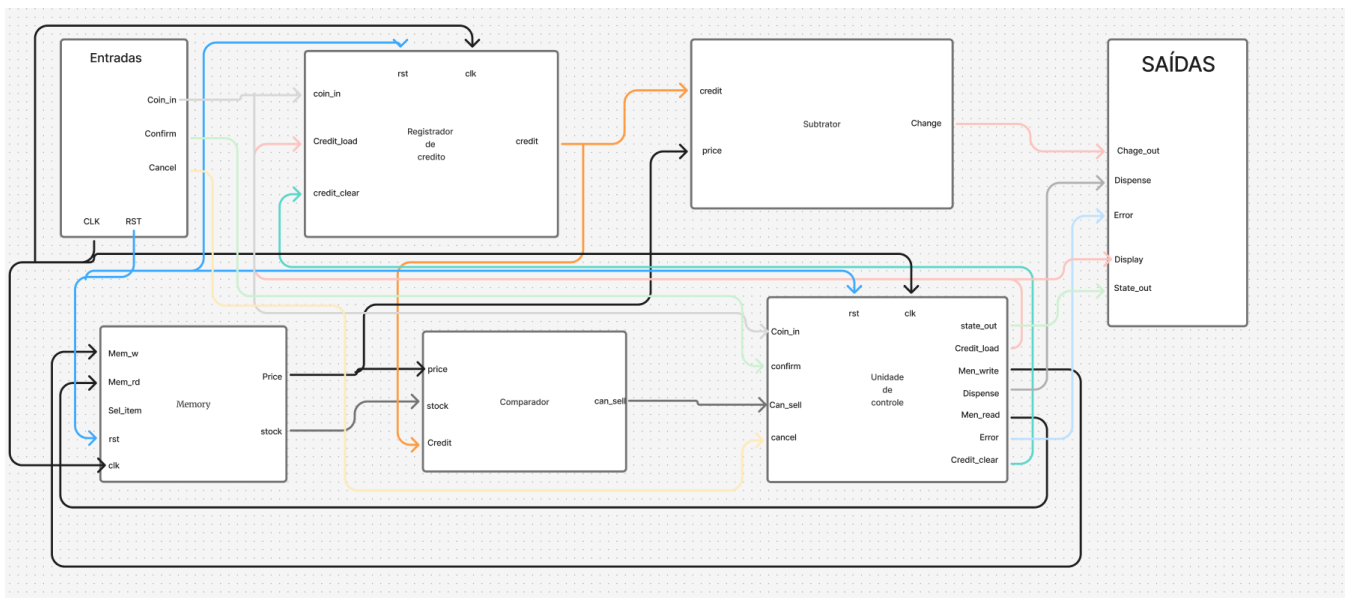


Figura 1 - Diagrama de blocos do controlador da vending machine

A unidade de controle gera os sinais `credit_load`, `credit_clear`, `mem_read` e `mem_write`. O registrador de crédito converte `coin_in` para 0, 25, 50 ou 100 centavos e acumula o valor. A memória fornece `price` e `stock` de forma síncrona. O comparador produz `can_sell`, enquanto o subtrator calcula `credit - price`. No top-level, `display` replica o crédito e `change_out` é registrado para venda, erro ou cancelamento.

3. Descrição dos módulos

Módulo	Função e decisão de projeto
vending_pkg.v	Parâmetros, preços, estoques, codificação dos seis estados e função coin_to_value.
registrador_credito.v	Registrador síncrono de 8 bits. Reset/clear têm prioridade sobre a acumulação.
memory.v	Memória 4 x 16 bits. Leitura e decremento de estoque são síncronos; preços permanecem fixos.
Comparador.v	Lógica combinacional que valida simultaneamente crédito suficiente e estoque diferente de zero.
subtrator.v	Lógica combinacional para cálculo do troco em centavos.
control_unit.v	FSM de Moore. O sinal check_ready mantém CHECK por um ciclo para respeitar a leitura síncrona.
vending_top.v	Integra os blocos, expõe as saídas e registra change_out.
tb_vending.v	Testbench self-checking com tarefas, timeout, waveform e quatro cenários obrigatórios.

4. Máquina de estados

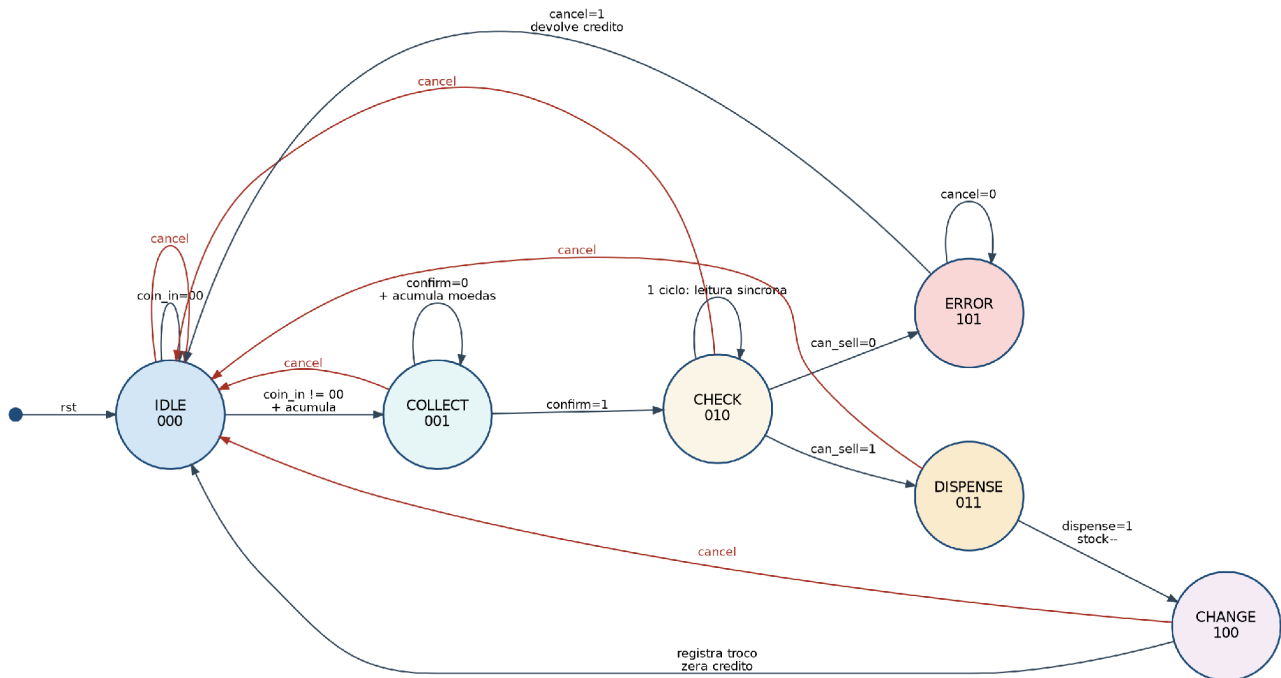


Figura 2 - Diagrama de transição da FSM. As transições vermelhas representam cancelamento com retorno ao IDLE.

Fluxo normal: IDLE -> COLLECT -> CHECK -> DISPENSE -> CHANGE -> IDLE. Fluxo de erro: CHECK -> ERROR; o estado ERROR aguarda cancel.

5. Metodologia de verificação

O testbench gera clock de 10 ns, aplica reset por dois ciclos e utiliza tarefas para inserir moedas, confirmar compras, cancelar operações, aguardar estados e comparar resultados. Cada verificação incrementa os contadores de PASS ou FAIL. Os sinais internos de crédito e estoque também são acessados hierarquicamente para confirmar o estado final do datapath.

#	Cenário	Estímulo	Resultado esperado
1	Compra com troco	Café de R\$0,25 pago com R\$1,00	dispense=1, change_out=75 e crédito final zero
2	Crédito insuficiente	Snack de R\$1,00 pago com R\$0,25	estado ERROR e error=1
3	Cancelamento	Duas moedas de R\$1,00 e cancelamento	retorno ao IDLE, crédito zero e devolução de 200
4	Estoque zerado	Cinco compras de café e uma sexta tentativa	estoque zero e ERROR na sexta compra

5.1 Saída do terminal - cenários 1 e 2

```
=====
CENARIO 1 - Compra com troco
=====
Cafe R$0,25 + moeda R$1,00 => troco R$0,75
Reset          estado=IDLE      credit=R$0,00 troco=R$0,00 disp=0 err=0

Compra: Cafe | preco=R$0,25 | moeda=R$1,00
Moeda: coin_in=11 (R$1,00)
Apos moeda     estado=COLLECT   credit=R$1,00 troco=R$0,00 disp=0 err=0
Confirmar: Cafe | preco=R$0,25
Apos confirm   estado=CHECK     credit=R$1,00 troco=R$0,00 disp=0 err=0
[PASS] C1: chegou em DISPENSE | estado=DISPENSE
Status        estado=DISPENSE  credit=R$1,00 troco=R$0,00 disp=1 err=0
[PASS] C1: dispense=1
[PASS] C1: chegou em CHANGE | estado=CHANGE
Status        estado=CHANGE    credit=R$1,00 troco=R$0,75 disp=0 err=0
[PASS] C1: change_out=75
[PASS] C1: voltou para IDLE | estado=IDLE
Status        estado=IDLE      credit=R$0,00 troco=R$0,75 disp=0 err=0
[PASS] C1: credito=0

=====
CENARIO 2 - Credito insuficiente
=====
Snack R$1,00 + moeda R$0,25 => ERROR
Reset          estado=IDLE      credit=R$0,00 troco=R$0,00 disp=0 err=0

Compra: Snack | preco=R$1,00 | moeda=R$0,25
Moeda: coin_in=01 (R$0,25)
Apos moeda     estado=COLLECT   credit=R$0,25 troco=R$0,00 disp=0 err=0
Confirmar: Snack | preco=R$1,00
Apos confirm   estado=CHECK     credit=R$0,25 troco=R$0,00 disp=0 err=0
[PASS] C2: chegou em ERROR | estado=ERROR
Status        estado=ERROR     credit=R$0,25 troco=R$0,00 disp=0 err=1
[PASS] C2: error=1
```

Figura 3 - Compra com troco e crédito insuficiente. Todas as checagens apresentadas foram aprovadas.

5.2 Saída do terminal - cenários 3 e 4

Parte 1

```

=====
CENARIO 3 - Cancelamento
=====
Inserir R$1,00 + R$1,00 e cancelar => devolucao R$2,00
Reset          estado=IDLE      credit=R$0,00 troco=R$0,00 disp=0 err=0
Moeda: coin_in=11 (R$1,00)
Apos moeda     estado=COLLECT    credit=R$1,00 troco=R$0,00 disp=0 err=0
Moeda: coin_in=11 (R$1,00)
Apos moeda     estado=COLLECT    credit=R$2,00 troco=R$0,00 disp=0 err=0
[PASS] C3: credito antes do cancel=200
Cancelamento solicitado.
Apos cancel    estado=IDLE      credit=R$0,00 troco=R$2,00 disp=0 err=0
[PASS] C3: credito=0
[PASS] C3: estado=IDLE
[PASS] C3: change_out=200
=====

CENARIO 4 - Estoque zerado
=====
Comprar cafe 5 vezes. Na 6a tentativa deve dar ERROR.
Reset          estado=IDLE      credit=R$0,00 troco=R$0,00 disp=0 err=0

Cafe 1/5 | estoque antes=5

Compra: Cafe | preco=R$0,25 | moeda=R$0,25
Moeda: coin_in=01 (R$0,25)
Apos moeda     estado=COLLECT    credit=R$0,25 troco=R$0,00 disp=0 err=0
Confirmar: Cafe | preco=R$0,25
Apos confirm   estado=CHECK      credit=R$0,25 troco=R$0,00 disp=0 err=0
[PASS] C4.1: DISPENSE | estado=DISPENSE
Status         estado=DISPENSE  credit=R$0,25 troco=R$0,00 disp=1 err=0
[PASS] C4.1: IDLE | estado=IDLE
Status         estado=IDLE      credit=R$0,00 troco=R$0,00 disp=0 err=0
[PASS] C4.1: estoque atualizado

```

Parte 2

```

Cafe 2/5 | estoque antes=4

Compra: Cafe | preco=R$0,25 | moeda=R$0,25
Moeda: coin_in=01 (R$0,25)
Apos moeda     estado=COLLECT    credit=R$0,25 troco=R$0,00 disp=0 err=0
Confirmar: Cafe | preco=R$0,25
Apos confirm   estado=CHECK      credit=R$0,25 troco=R$0,00 disp=0 err=0
[PASS] C4.2: DISPENSE | estado=DISPENSE
Status         estado=DISPENSE  credit=R$0,25 troco=R$0,00 disp=1 err=0
[PASS] C4.2: IDLE | estado=IDLE
Status         estado=IDLE      credit=R$0,00 troco=R$0,00 disp=0 err=0
[PASS] C4.2: estoque atualizado

Cafe 3/5 | estoque antes=3

Compra: Cafe | preco=R$0,25 | moeda=R$0,25
Moeda: coin_in=01 (R$0,25)
Apos moeda     estado=COLLECT    credit=R$0,25 troco=R$0,00 disp=0 err=0
Confirmar: Cafe | preco=R$0,25
Apos confirm   estado=CHECK      credit=R$0,25 troco=R$0,00 disp=0 err=0
[PASS] C4.3: DISPENSE | estado=DISPENSE
Status         estado=DISPENSE  credit=R$0,25 troco=R$0,00 disp=1 err=0
[PASS] C4.3: IDLE | estado=IDLE
Status         estado=IDLE      credit=R$0,00 troco=R$0,00 disp=0 err=0
[PASS] C4.3: estoque atualizado

Cafe 4/5 | estoque antes=2

Compra: Cafe | preco=R$0,25 | moeda=R$0,25
Moeda: coin_in=01 (R$0,25)
Apos moeda     estado=COLLECT    credit=R$0,25 troco=R$0,00 disp=0 err=0
Confirmar: Cafe | preco=R$0,25
Apos confirm   estado=CHECK      credit=R$0,25 troco=R$0,00 disp=0 err=0
[PASS] C4.4: DISPENSE | estado=DISPENSE
Status         estado=DISPENSE  credit=R$0,25 troco=R$0,00 disp=1 err=0
[PASS] C4.4: IDLE | estado=IDLE
Status         estado=IDLE      credit=R$0,00 troco=R$0,00 disp=0 err=0
[PASS] C4.4: estoque atualizado

```

Figura 4/5 - Cancelamento com devolução de R\$2,00, esgotamento do café e resumo final: PASS=31 e FAIL=0.

Parte 3

```

Cafe 5/5 | estoque antes=1

Compra: Cafe | preco=R$0,25 | moeda=R$0,25
Moeda: coin_in=01 (R$0,25)
Apos moeda     estado=COLLECT    credit=R$0,25 troco=R$0,00 disp=0 err=0
Confirmar: Cafe | preco=R$0,25
Apos confirm   estado=CHECK      credit=R$0,25 troco=R$0,00 disp=0 err=0
[PASS] C4.5: DISPENSE | estado=DISPENSE
Status         estado=DISPENSE  credit=R$0,25 troco=R$0,00 disp=1 err=0
[PASS] C4.5: IDLE | estado=IDLE
Status         estado=IDLE      credit=R$0,00 troco=R$0,00 disp=0 err=0
[PASS] C4.5: estoque atualizado
[PASS] C4: estoque=0

Tentando 6a compra de cafe...

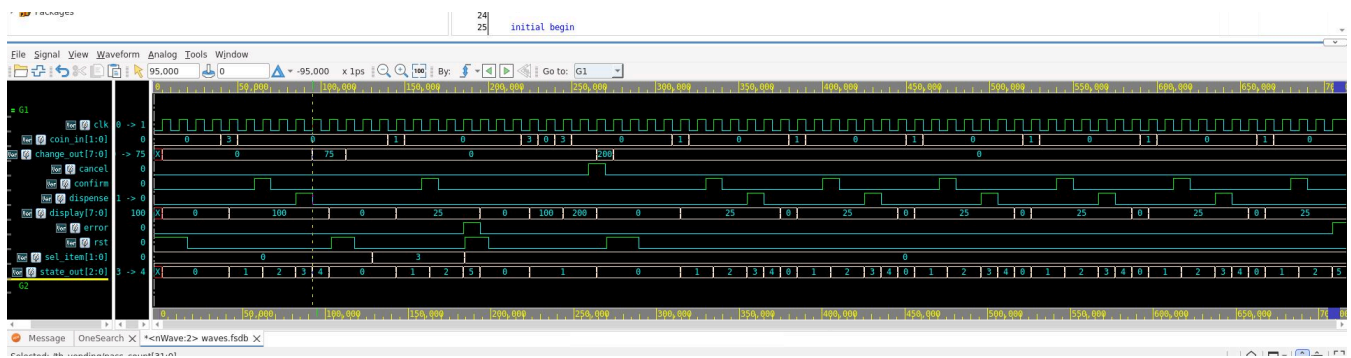
Compra: Cafe | preco=R$0,25 | moeda=R$0,25
Moeda: coin_in=01 (R$0,25)
Apos moeda     estado=COLLECT    credit=R$0,25 troco=R$0,00 disp=0 err=0
Confirmar: Cafe | preco=R$0,25
Apos confirm   estado=CHECK      credit=R$0,25 troco=R$0,00 disp=0 err=0
[PASS] C4.6: chegou em ERROR | estado=ERROR
Status         estado=ERROR     credit=R$0,25 troco=R$0,00 disp=0 err=0
[PASS] C4.6: error=1
[PASS] C4.6: estoque permaneceu 0

=====
RESUMO
=====
PASS=31 | FAIL=0
RESULTADO FINAL: TODOS OS TESTES PASSARAM
$finish called from file "sim/tb_vending.sv", line 251.
$finish at simulation time          706000
V C S   S i m u l a t i o n   R e p o r t

```

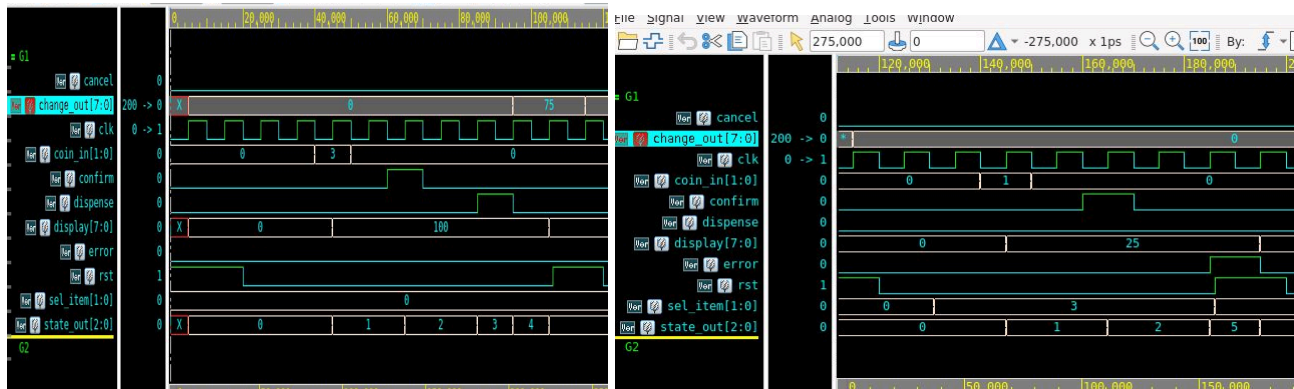
Resultado da simulação: os 31 testes passaram, sem falhas. O comportamento observado confirma as transições da FSM, o pulso de dispense, o cálculo e a devolução do troco, o clear do crédito e a proteção contra decremento abaixo de zero.

5.3 Waveforms anotadas



Cenário 1

Cenário 2



Cenário 3



Cenário 4

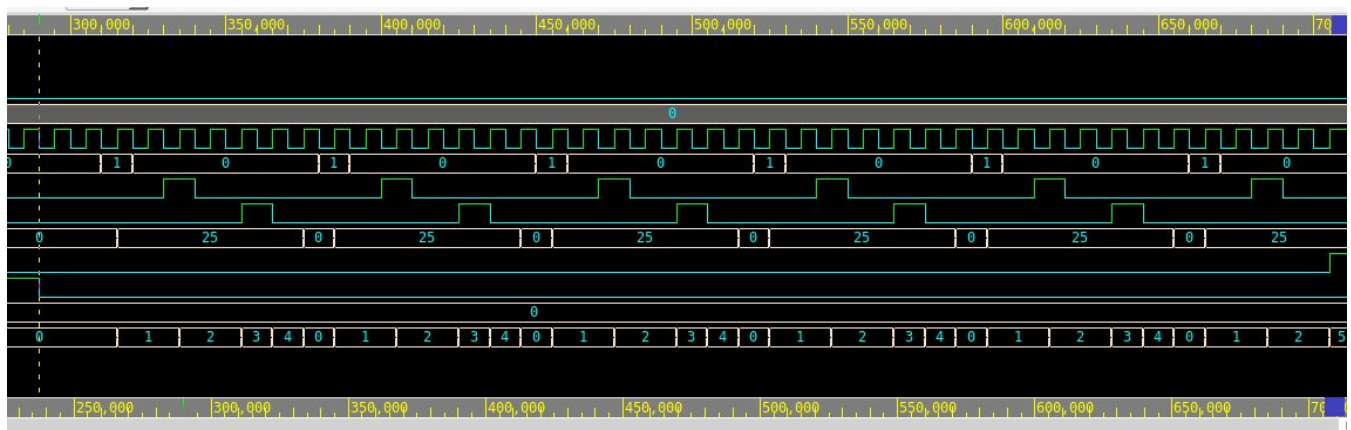


Figura 5 - Waveforms dos quatro cenários obrigatórios, separadas e identificadas por painel.

- Cenário 1: IDLE-COLLECT-CHECK-DISPENSE-CHANGE-IDLE, dispense=1 e change_out=75.

- Cenário 2: após CHECK, state_out=5 e error=1.
- Cenário 3: cancel retorna a IDLE, zera display e registra change_out=200.
- Cenário 4: cinco vendas válidas; na sexta tentativa a FSM entra em ERROR.

Codificação de state_out: 0=IDLE, 1=COLLECT, 2=CHECK, 3=DISPENSE, 4=CHANGE e 5=ERROR.

6. Síntese lógica e constraints

A síntese foi executada no Synopsys Design Compiler X-2025.06-SP2, utilizando a biblioteca SAED32 RVT no canto típico tt, 1,05 V e 25 °C. O script analisa os arquivos SystemVerilog, elabora vending_top, executa link, lê o arquivo SDC, roda check_design e aplica compile_ultra. Ao final são exportados a netlist, o arquivo DDC e os relatórios de área, timing, potência e violações.

6.1 Restrições principais

Restrição	Valor
Clock inicial	20 ns (50 MHz)
Incerteza de setup	0,5 ns
Incerteza de hold	0 ns
Input delay	3 ns para coin_in, sel_item, confirm, cancel e rst
Output delay	3 ns para todas as saídas
Carga de saída	0,05 pF
Fanout máximo	8

6.2 Exploração do período de clock

Período (ns)	Frequência (MHz)	Slack (ns)	Área total	Timing
20	50,00	15,51	958,14	Atendido
18	55,56	13,51	958,14	Atendido
16	62,50	11,51	958,14	Atendido
14	71,43	9,51	958,14	Atendido
12	83,33	7,51	958,14	Atendido
10	100,00	5,51	958,14	Atendido
8	125,00	3,51	958,14	Atendido
6	166,67	1,51	958,14	Atendido
4	250,00	-0,11	1264,60	Violado

O menor período testado com slack não negativo foi 6 ns, equivalente a 166,67 MHz. Assim, 166,67 MHz é a maior frequência validada no sweep de 2 ns. Em 4 ns (250 MHz), o slack passou a -0,11 ns e o circuito deixou de atender ao setup.

7. Análise dos resultados de síntese

7.1 Área e potência

De 20 ns até 6 ns, a área permaneceu em 958,14. Ao impor 4 ns, o Design Compiler aumentou a área para 1264,60, crescimento de aproximadamente 31,98%. O número de células combinacionais passou de 202 para 331 e o número de buffers/inversores de 55 para 96. Essa troca de área por velocidade indica uso de células e redes mais agressivas para reduzir atrasos, porém ainda insuficientes para fechar o timing.

A potência total estimada subiu de 158,70 μ W em 6 ns para 229,83 μ W em 4 ns, aumento aproximado de 44,82%. Esse comportamento é compatível com a maior frequência de clock e com a implementação de maior área.

7.2 Caminho crítico e estado associado

Período	Startpoint	Endpoint	Slack	Interpretação
6 ns	coin_in[1]	credit_inst/credit_reg[7]	+1,51 ns	Conversão da moeda e soma no registrador de crédito.
4 ns	sel_item[1]	memoria_inst/mem_reg[3] [4]	-0,11 ns	Seleção do item e lógica de decremento/escrita do estoque.

No ponto de operação válido de 6 ns, o caminho crítico atravessa o registrador de crédito. Ele começa em coin_in[1], passa pela decodificação do valor da moeda e pela lógica do somador, chegando ao bit 7 de credit. Funcionalmente, esse caminho está associado aos estados IDLE/COLLECT, nos quais credit_load permite a acumulação de moedas.

Quando a restrição é reduzida para 4 ns, o caminho mais crítico migra para a memória: sel_item seleciona a palavra e a lógica combinacional prepara a atualização do estoque antes de mem_reg. Esse caminho está associado ao estado DISPENSE, no qual mem_write é ativado e stock é decrementado.

7.3 Comparação de autoungroup

Comando em 6 ns	Área	Slack	Resultado
compile_ultra	958,139794	+1,51 ns	Atendido
compile_ultra -no_autoungroup	958,139794	+1,51 ns	Atendido

Não houve diferença mensurável de área, slack ou caminho crítico. Para este design pequeno e bem modularizado, preservar a hierarquia não impediu o compilador de obter a mesma implementação física estimada.

8. Proposta de otimização do RTL

Uma modificação possível para reduzir o caminho crítico sem alterar a interface funcional é representar internamente os valores em unidades de R\$0,25. Nesse formato, as moedas seriam codificadas como 0, 1, 2 e 4; os preços como 1, 2, 3 e 4; e o crédito poderia usar apenas 4 bits. As saídas display e change_out seriam convertidas novamente para centavos por multiplicação constante por 25.

Essa mudança preserva o comportamento externo e reduz a largura do somador, do comparador e do subtrator. Como todos os preços e moedas do projeto são múltiplos de 25 centavos, não há perda de informação. Outra melhoria seria separar explicitamente o datapath de atualização da memória e avaliar o registro de sel_item na confirmação da compra, reduzindo a dependência de uma entrada externa durante DISPENSE.

9. Conclusão

O controlador foi implementado com sucesso e atendeu aos requisitos funcionais. A FSM de Moore coordenou corretamente a coleta de moedas, a leitura síncrona da memória, a validação da compra, a liberação do produto, o cálculo do troco e o tratamento de cancelamentos e erros. O uso de check_ready foi essencial para aguardar a disponibilidade de price e stock antes de avaliar can_sell.

A verificação automática executou 31 checagens, todas aprovadas. Os quatro cenários obrigatórios foram observados tanto no terminal quanto nas waveforms. Na síntese, o design fechou timing até 6 ns no sweep realizado, correspondendo a 166,67 MHz, com área total de 958,14 e slack de 1,51 ns. Em 4 ns, mesmo com aumento de aproximadamente 32% na área, ocorreu violação de -0,11 ns.

As principais dificuldades estiveram na sincronização entre FSM e memória, no registro correto do troco e na exploração sistemática das constraints. Como aprendizado, o projeto demonstrou a importância da separação entre controle e datapath, da verificação self-checking e da análise conjunta de área, timing e potência durante a síntese.

10. Arquivos entregues

vending_pkg.sv, registrador_credito.sv, memory.sv, Comparador.sv, subtrator.sv, control_unit.sv, vending_top.sv, tb_vending.sv, Makefile, synth.tcl, vending.sdc, relatórios de síntese e este relatório.

Repositório: <https://github.com/ubajjara/Machine.git>